# ⏱ DAY 2: COMPLETE 6-HOUR MASTER TRAINER PLAYBOOK

## Traditional Machine Learning & Parallel Computing using Intel oneAPI

**Course Code:** `GLA-HPC-BCSG701` | **Track:** Intel & NEC Supercomputing Alliance
**Lead Trainer:** Shivam Gupta | **Daily Duration:** 6 Full Hours (Session 1: Morning 3h + Session 2: Afternoon 3h)
**Official Syllabus Reference:** `Intel & NEC - High Performance Computing.pdf` (Page 2)
**Day 2 Complete Terminology Glossary:** `00_DAY_02_MASTER_TERMINOLOGY_AND_CONCEPT_GLOSSARY.md`
**Day 2 Hands-On Lab Runbook:** `02_DAY_02_HANDS_ON_LAB_AND_COMMANDS.md`

---

## 🎯 1. Official Syllabus Scope (Page 2 of PDF — Strictly Preserved)

The official curriculum specifies **strictly 5 core topics for Day 2**:

1. **Parallel Computing** (Amdahl's law, serial bottleneck, task vs data parallelism)
2. **OpenMP for multithreading, MPI for distributed systems**
3. **Data decomposition and synchronization**
4. **Using Intel AI Analytics Toolkit (Optimized Scikit-learn, Pandas, NumPy)**
5. **Intel Extension for Scikit-learn for optimized ML algorithms**

> 📌 **IMPORTANT**
> **Curriculum Boundary Discipline:** Deep Learning (TensorFlow-MKL, oneDNN, Horovod) and Slurm workload management belong strictly to Days 3 and 4. Generative AI and LLMs belong to Day 5. Keep Day 2 laser-focused on multi-core parallelism (OpenMP), cluster message passing (MPI), and classic ML acceleration (oneDAL).

---

## ⏰ 2. Complete 6-Hour Dual-Session Master Daily Schedule

```
+-------------------------------------------------------------------------------------+
|                    DAY 2 MASTER 6-HOUR SCHEDULE (09:30 AM - 04:30 PM)               |
+-------------------------------------------------------------------------------------+
| 👨‍🏫 SESSION 1: MORNING THEORY & ARCHITECTURE MASTERCLASS (09:30 AM - 12:30 PM | 3 Hours)  |
|    "Parallel Mathematics → OpenMP & MPI Primitives → oneDAL Architecture"           |
+-----------------------+-------------------------------------------------------------+
|  09:30 - 10:15 (45m)  | PART 1: Parallel Computing Fundamentals & Amdahl's Law      |
|                       |        (Moore's Law Wall, Task vs Data Parallelism, Math Formula) |
+-----------------------+-------------------------------------------------------------+
|  10:15 - 11:00 (45m)  | PART 2: Shared-Memory Multithreading with OpenMP            |
|                       |        (Fork-Join Model, Reduction Clause, False Sharing Physics) |
+-----------------------+-------------------------------------------------------------+
|  11:00 - 11:10 (10m)  | ☕ REJUVENATION BREAK (10 Minutes)                           |
+-----------------------+-------------------------------------------------------------+
|  11:10 - 11:45 (35m)  | PART 3: Distributed Systems with MPI                        |
|                       |        (Ranks, Communicators, Point-to-Point vs Collective Trees) |
+-----------------------+-------------------------------------------------------------+
|  11:45 - 12:15 (30m)  | PART 4: Intel AI Analytics Toolkit & oneDAL Acceleration    |
|                       |        (NumPy MKL, Modin Pandas, sklearnex patch_sklearn() 20x)  |
+-----------------------+-------------------------------------------------------------+
|  12:15 - 12:30 (15m)  | PART 5: Morning Review, 5 Rapid Check Questions & Lunch Teaser |
+-----------------------+-------------------------------------------------------------+
| 🍱 LUNCH & INTERMISSION BREAK (12:30 PM - 01:30 PM | 60 Minutes)                    |
|    Workstation core allocation audit, Google Colab runtime pre-warming              |
+-------------------------------------------------------------------------------------+
| 💻 SESSION 2: AFTERNOON HANDS-ON SUPERCOMPUTER LAB (01:30 PM - 04:30 PM | 3 Hours)      |
|    "OpenMP Benchmarks → MPI Ranks → 500k Sample K-Means & Random Forest → 25 MCQs"  |
+-----------------------+-------------------------------------------------------------+
|  01:30 - 02:15 (45m)  | STAGE 0 & 1: OpenMP Vector Reduction & Thread Scaling Benchmark |
|                       |        (Compiling with -fopenmp, 1, 2, 4, 8 threads speedup test) |
+-----------------------+-------------------------------------------------------------+
|  02:15 - 02:45 (30m)  | STAGE 2: OpenMP False Sharing Elimination & MESI Protocol   |
|                       |        (Demonstrating 4x to 8x speedup using alignas(64) padding) |
+-----------------------+-------------------------------------------------------------+
|  02:45 - 03:30 (45m)  | STAGE 3: C++ MPI Distributed Communication Across Processes |
|                       |        (Compiling with mpicxx, running 4 ranks, MPI_Reduce test) |
+-----------------------+-------------------------------------------------------------+
|  03:30 - 04:10 (40m)  | STAGE 4 & 5: Intel AI Analytics Toolkit & oneDAL ML Acceleration |
|                       |        (Benchmarking 500k K-Means & Random Forest: 15x-35x speedup)|
+-----------------------+-------------------------------------------------------------+
|  04:10 - 04:30 (20m)  | ASSESSMENT & WRAP-UP: 25 Evaluated MCQs & Checklist Sign-Off |
|                       |        (22-Point Mastery Checklist, 25 MCQs, Day 3 Preview) |
+-------------------------------------------------------------------------------------+
```

---

## 🎙️ 3. Line-by-Line Minute-by-Minute Trainer Delivery Script (Session 1: Morning)

### PART 1: Parallel Computing Fundamentals (09:30 - 10:15 | 45 Mins)

- **Slide Alignment:** Day 2 Slide Deck (Slides 1–5: Introduction to Parallelism & Amdahl's Law).
- **Concept:** Breaking the serial bottleneck; mathematical bounds on multi-core speedup.
- **Trainer Script (Hinglish/English):**

> "Good morning, class! Yesterday we mastered the Linux operating system, remote SSH keys, and initialized our Intel oneAPI toolchains. Today on Day 2, we tackle the core physics of why High Performance Computing exists: **breaking the serial bottleneck**. Why can't Intel build a single 25 GHz CPU core? Because of the **Power Wall** ( $P = C \cdot V^2 \cdot f$)—a chip running at 25 GHz would melt its silicon package. Instead of one ultra-fast core, modern Intel Xeon processors feature **32, 64, or 128 physical cores**. But here is the catch: if you write standard serial Python or C++ code, your program runs on **ONLY ONE CORE**, leaving the other 63 cores completely idle! Look at **Amdahl's Law**:
>
> $$\text{Speedup} = \frac{1}{(1 - P) + \frac{P}{N}}$$
>
> If 10% of your program is serial ($1 - P = 0.10$), even if I give you 10,000 Xeon cores ($N \to \infty$), your theoretical maximum speedup can **NEVER exceed 10x**! That is why we must learn data decomposition and parallel programming."

- **Whiteboard Derivation 1:**

```
Amdahl's Speedup Curve:
Speedup
  ^
10|-------------------------- Maximum Ceiling S_max = 1 / (1 - P) = 10x
  |                  . . . . .
  |            . . .
  |       . .
  |   . .
  | .
  +----------------------------> Core Count (N)
  1    4    16   64   256  1024
```

---

## PART 2: Shared-Memory Multithreading with OpenMP (10:15 - 11:00 | 45 Mins)

- **Slide Alignment:** Day 2 Slide Deck (Slides 6–10: OpenMP Fork-Join & Memory Primitives).
- **Concept:** Fork-join execution, compiler pragmas, reduction clauses, and false sharing.
- **Trainer Script:**

> "Now let's see how we parallelize code on a single multi-core server using **OpenMP**. OpenMP uses simple compiler directives: `#pragma omp`. In the **Fork-Join Model**, the program begins as a single master thread. When it hits `#pragma omp parallel for`, the master thread **forks** a team of worker threads across CPU cores. But watch out for **Race Conditions**! If two threads execute `total_sum += a[i]` simultaneously, they overwrite each other's register values, corrupting the final answer. To solve this with zero locking overhead, we use the **reduction clause**: `#pragma omp parallel for reduction(+:total_sum)` Each thread gets a private accumulator in its L1 cache, and OpenMP combines them in a tree reduction at the end. Now look at **False Sharing**: if two threads write to adjacent array elements that sit on the same 64-byte cache line, the CPU cache coherence protocol (MESI) bounces that cache line back and forth between cores, dropping performance by 80x!"

- **Live Whiteboard Sketch:**

```
Cache Line Invalidation (False Sharing):
+-------------------------------------------------------+
|             64-BYTE L1 DATA CACHE LINE                |
|  Thread 0 writes: array[0]  |  Thread 1 writes: array[1]  |
+-----------------------------+-----------------------------+
             ^                               ^
       Core 0 (L1 Cache)              Core 1 (L1 Cache)
             |                               |
             +=== MESI Bus Invalidation Cycle ===+ (Latency: 80ns per write!)
```

---

## ☕ BREAK (11:00 - 11:10 | 10 Minutes)

---

## PART 3: Distributed Systems with MPI (11:10 - 11:45 | 35 Mins)

- **Slide Alignment:** Day 2 Slide Deck (Slides 11–14: Distributed Systems & MPI Architecture).
- **Concept:** Message passing across independent servers, ranks, point-to-point, and collective trees.
- **Trainer Script:**

> "Welcome back. OpenMP works on a single computer where all cores share the same RAM. But what happens when your dataset is 5 Terabytes and exceeds one server's memory? You must distribute the job across multiple separate compute nodes connected by an InfiniBand network. These nodes **do not share memory**. Here, we use **MPI (Message Passing Interface)**. In MPI:
>
> - Each process has its own private memory and a unique ID called a **Rank** ( `0, 1, 2, ... N-1` ).
> - Point-to-Point: `MPI_Send` and `MPI_Recv` between two ranks.
> - Collectives: `MPI_Bcast` (one sends to all), `MPI_Scatter` (splits data among ranks), and `MPI_Reduce` (aggregates partial sums from all ranks into Rank 0). Today we demonstrate how 4 simulated MPI ranks compute a distributed sum reduction in parallel!"

---

## PART 4: Intel AI Analytics Toolkit & oneDAL ML Acceleration (11:45 - 12:15 | 30 Mins)

- **Slide Alignment:** Day 2 Slide Deck (Slides 15–20: AI Kit, oneDAL, and sklearnex).
- **Concept:** Zero-code-change Scikit-learn acceleration via Intel oneDAL dynamic monkey patching.
- **Trainer Script:**

> "Now we transition into the AI layer. Standard Scikit-learn is single-threaded and struggles on large datasets because Python is bound by the Global Interpreter Lock (GIL). Intel created the **AI Analytics Toolkit** containing **Intel Extension for Scikit-learn (`sklearnex`)**. Under the hood, `sklearnex` calls **Intel oneDAL**—a high-performance C++ library compiled with AVX-512 vectorization and cache-conscious matrix operations. How do you use it? Look at these two lines of code:
>
> ```
> from sklearnex import patch_sklearn
> patch_sklearn()
> ```
>
> That's it! Zero changes to your Scikit-learn model code. When you call `kmeans.fit(X)` or `RandomForestClassifier.fit(X)`, it executes in parallel across all 64 Xeon cores. In our afternoon lab right now, we will train a K-Means model on **500,000 samples with 32 features** and compare stock Scikit-learn vs. Intel oneDAL. You will see a **15× to 35× measured speedup** right on your screen!"

---

### PART 5: Morning Review, 5 Rapid Check Questions & Lunch Teaser (12:15 - 12:30 | 15 Mins)

**[12:15 - 12:25] 5-Question Quick Classroom Check**

1. *Q1: State Amdahl's Law and explain what limits speedup as core count approaches infinity.*
   **Answer:** $S = 1/((1-P) + P/N)$. The serial portion $(1-P)$ establishes the theoretical maximum ceiling $1/(1-P)$.
2. *Q2: What is the primary difference between shared-memory (OpenMP) and distributed-memory (MPI) programming models?*
   **Answer:** OpenMP threads share the same physical address space on a single node; MPI processes have isolated address spaces and communicate via explicit network messages.
3. *Q3: What causes False Sharing in multi-threaded C++ programs?*
   **Answer:** Multiple threads modifying independent variables that reside on the same 64-byte CPU cache line, triggering constant MESI bus invalidation.
4. *Q4: What two lines of Python code dynamically patch Scikit-Learn with Intel oneDAL?*
   **Answer:** `from sklearnex import patch_sklearn; patch_sklearn()`.
5. *Q5: Which MPI collective operation aggregates partial values from all ranks into a single final result on Rank 0?*
   **Answer:** `MPI_Reduce`.

**[12:25 - 12:30] Lunch Break Announcement & Afternoon Lab Setup**

- **What Trainer Shivam Says:**

> "Outstanding work this morning! We have mastered the mathematics of parallel speedup, OpenMP pragmas, MPI collectives, and oneDAL acceleration. We now take our 1-hour lunch break until 01:30 PM. In our afternoon session, we go straight to the servers! We will compile and benchmark OpenMP vector reductions, witness false sharing destruction and fix it, run simulated MPI message passing, and run real 500,000-sample Machine Learning benchmarks measuring 20x speedup. See you at 01:30 PM sharp!"

---

## 🍱 INTERMISSION: Lunch & Cluster Preflight (12:30 PM - 01:30 PM | 60 Mins)

- **Trainer System Checklist during break:**
  1. Verify GCC compiler with `-fopenmp` flag support on all student workstations.
  2. Test `pip install scikit-learn-intelex` in a fresh test shell.

3. Ensure OpenMPI or MPICH runtime is active ( `mpirun --version` ).
4. Project Stage 0 environment preflight verification on the main screen.

---

## 💻 4. SESSION 2: Afternoon Hands-On Supercomputer Lab Playbook (01:30 PM - 04:30 PM | 3 Hours)

*Reference Runbook:* `02_DAY_02_HANDS_ON_LAB_AND_COMMANDS.md`
*Notebook Reference:* `Day_02_Traditional_ML_oneAPI_OpenMP_MPI_oneDAL.ipynb`

```
+-------------------------------------------------------------------------------+
|                   SESSION 2: 3-HOUR HANDS-ON LAB EXECUTION PHASES             |
+-------------------------------------------------------------------------------+
| 01:30 - 02:15 (45m) | Phase 1: OpenMP Vector Reduction & Thread Scaling (Stages 0 & 1)   |
| 02:15 - 02:45 (30m) | Phase 2: False Sharing Elimination & MESI Protocol Fix (Stage 2)   |
| 02:45 - 03:30 (45m) | Phase 3: C++ MPI Distributed Communication Across 4 Ranks (Stage 3)|
| 03:30 - 04:10 (40m) | Phase 4: oneDAL 500k Sample K-Means & Random Forest (Stages 4 & 5) |
| 04:10 - 04:30 (20m) | Phase 5: Student 25 MCQs Assessment & 22-Point Checklist Sign-Off  |
+-------------------------------------------------------------------------------+
```

### [01:30 - 02:15] Phase 1: OpenMP Vector Reduction & Thread Scaling (Stages 0 & 1)

- **Trainer Instruction (To Class):**

> "Welcome back! Open your terminal or the Day 2 Google Colab notebook. Let's start with Stage 1: We are writing `openmp_vector_sum.cpp` to sum 50 million double-precision numbers. Compile it with: `g++ -O3 -fopenmp openmp_vector_sum.cpp -o openmp_vector_sum`. Now run it with 1 thread: `OMP_NUM_THREADS=1 ./openmp_vector_sum`. Then scale to 2, 4, and 8 threads. Look at your execution time: notice how the time drops, but speedup sub-linearly tapers off as predicted by Amdahl's Law!"

- **Trainer Walkabout Check:**
  - Verify students correctly set `export OMP_NUM_THREADS=4`.
  - Ensure students do not omit `-fopenmp` in the compiler flags.

### [02:15 - 02:45] Phase 2: False Sharing Elimination & MESI Protocol Fix (Stage 2)

- **Trainer Instruction (To Class):**

> "Now look at Stage 2: `false_sharing_demo.cpp`. In version 1, four threads write to `struct ThreadState { int count; } state[4];`. All 4 counts sit inside a single 64-byte cache line! Run it and measure the execution time. In version 2, we apply `alignas(64) int count;`. Now each thread's variable is isolated onto its own independent 64-byte cache line. Run version 2. You will observe an immediate **4x to 8x performance improvement** with zero algorithmic changes! That is hardware-aware programming."

### [02:45 - 03:30] Phase 3: C++ MPI Distributed Communication Across 4 Ranks (Stage 3)

- **Trainer Instruction (To Class):**

> "In Stage 3, we leave the single server and simulate distributed computing across separate processes. Look at `mpi_collective_reduction.cpp`. Compile with: `mpicxx -O3 mpi_collective_reduction.cpp -o mpi_collective_reduction`. Launch 4 MPI worker ranks: `mpirun --allow-run-as-root -np 4 ./mpi_collective_reduction`. Notice the output: Rank 0 initializes data, `MPI_Scatter` partitions the array, each rank computes its local sum, and `MPI_Reduce` aggregates the partial sums back into Rank 0!"

### [03:30 - 04:10] Phase 4: Intel AI Analytics Toolkit & oneDAL ML Acceleration (Stages 4 & 5)

- **Trainer Instruction (To Class):**

> "Now for our flagship Machine Learning benchmark. Open cell 7 or execute `intel_scikit_speedup.py`. We generate a massive synthetic dataset: **500,000 samples with 32 continuous features**. First, we train standard Scikit-Learn KMeans. Note down the execution time (~18.5 seconds). Next, we enable `sklearnex.patch_sklearn()` and train identical KMeans on identical data. Look at the screen: execution completes in **~0.9 seconds! That is a 20.5x measured speedup** running live on your machine!"

- **Student Milestone Challenge:**
  - Have all 60 students post their measured speedup multiplier in the classroom chat!

### [04:10 - 04:30] Phase 5: Student 25 MCQs Assessment & Checklist Sign-Off

- **Trainer Activity:**
  1. Open `03_DAY_02_STUDENT_CHEATSHEET_AND_MCQS.md`.
  2. Review the 25 Evaluated MCQs. Walk through the explanations for questions on Amdahl's Law, False Sharing, and `sklearnex`.
  3. Verify all students complete the 22-point Day 2 Mastery Checklist.
  4. **Day 3 Teaser Announcement:**

> "Outstanding work today! You broke the serial bottleneck with OpenMP, scaled across processes with MPI, and accelerated Machine Learning by 20x with Intel oneDAL. Tomorrow on Day 3, we enter Deep Learning on HPC clusters! We will study Intel-optimized TensorFlow, oneDNN hardware operator fusion, Habana Gaudi 2/3 accelerators, Slurm batch scheduling, and distributed training with Horovod. Be here at 09:30 AM sharp!"

---

## 🛠️ 5. Top 5 Student Gotchas & 30-Second Rapid Fixes

| Issue Encountered | Why It Happened | 30-Second Trainer Fix |
|---|---|---|
| `undefined reference to 'omp_get_thread_num'` | Forgot `-fopenmp` compiler flag | Add `-fopenmp` to `g++` or `icpx` compilation command |
| OpenMP loop produces random/incorrect total sum | Race condition on shared accumulator | Add `reduction(+:variable_name)` clause to `#pragma omp parallel for` |
| MPI execution fails: `mpirun has detected an attempt to run as root` | Running inside Docker/Colab root shell | Append `--allow-run-as-root` flag to `mpirun` command |
| `patch_sklearn()` shows no speedup | Called `patch_sklearn()` AFTER importing models | Ensure `patch_sklearn()` is invoked before importing any `sklearn` classes |

| Issue Encountered | Why It Happened | 30-Second Trainer Fix |
| --- | --- | --- |
| Segment fault during 50M array allocation | Stack overflow due to stack array allocation | Allocate large arrays on heap using `new double[N]` or `std::vector<double>` |